



(12) 发明专利申请

(10) 申请公布号 CN 119988236 A

(43) 申请公布日 2025. 05. 13

(21) 申请号 202510451127.3

G06N 3/126 (2023.01)

(22) 申请日 2025.04.11

(71) 申请人 徐州工程学院

地址 221018 江苏省徐州市云龙区丽水路2号徐州工程学院

(72) 发明人 陶磊 党向盈 黄炳森 陈天昊
李鑫杰 高月广 吴志昊

(74) 专利代理机构 苏州大成君合知识产权代理
事务所(普通合伙) 32547

专利代理师 张印铎

(51) Int. Cl.

G06F 11/3668 (2025.01)

G06N 3/0464 (2023.01)

G06N 3/086 (2023.01)

G06N 3/096 (2023.01)

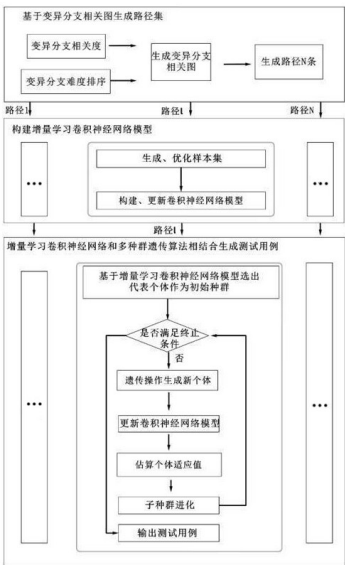
权利要求书3页 说明书12页 附图7页

(54) 发明名称

卷积神经网络结合遗传算法的智能装备软件测试方法

(57) 摘要

本发明公开了一种卷积神经网络结合遗传算法的智能装备软件测试方法,旨在将变异测试问题转化为传统覆盖路径测试问题,并生成有效测试用例。首先计算变异分支覆盖难度,构建相关度矩阵并生成变异分支相关图,再生成可执行路径集;然后针对多路径构建多任务测试用例生成数学模型;随后构建基于增量学习的卷积神经网络模型,进而预测高适应度个体作为多种群遗传方法的初始种群,最终借助基于增量学习卷积神经网络增强的多种群遗传方法,高效生成具备缺陷检测能力的测试用例。本发明生成具有高缺陷检测能力的测试用例,有效提升智能装备软件测试效率与质量。



1.一种卷积神经网络结合遗传算法的智能装备软件测试方法,其特征在于,包括以下步骤:

S1:基于变异分支相关图生成路径集;

S2:基于生成的路径集构建测试用例生成优化模型;

S3:利用所述优化模型按照预设规则构建增量学习卷积神经网络模型;

S4:将所述增量学习卷积神经网络和多种群遗传方法结合生成测试用例。

2.根据权利要求1所述的卷积神经网络结合遗传算法的智能装备软件测试方法,其特征在于,所述步骤S3中预设规则被实施为:

S3.1:优化初始样本集 $N^0 = \{X^0, P(X^0), Fit(X^0)\}$, $X^0 = \{x_1^0, x_2^0, \dots, x_{n'}^0\}$ 为初始的数据集, n' 为初始样本数量; $P(X^0)$ 为基于初始的数据集获得的对应的穿越路径集合; $Fit(X^0)$ 为 X^0 对应的适应值集合;

S3.1.1:针对初始穿越路径集 $P(X^0)$ 中每一条路径 $p_1(x_1^0), p_2(x_2^0), \dots, p_{n'}(x_{n'}^0)$, 获得哈希编码路径 $h_1, h_2 \dots h_{n'}$, 哈希编码路径集合为 $H = \{h_1, h_2 \dots h_{n'}\}$; 设 $z_i^0 = (x_i^0, p_i(x_i^0))$ 为一个输入特征值, 由它转化为哈希特征值 $NH_i = (x_i^0, h_i)$;

S3.1.2:将 $NH_1 = (x_1^0, h_1)$ 入栈 Sta 中; 栈外的 $NH_2 = (x_2^0, h_2)$ 是否能入栈 Sta , 满足下面两个条件之一:

(I) 将 h_2 与栈内的 h_1 进行比较, 如果 h_2 和 h_1 哈希编码不相同, 直接将 NH_2 放入 Sta 中;

(II) 如果 h_2 和 h_1 哈希编码相同, 则继续比较 x_1^0 和 x_2^0 , 如果 x_1^0 和 x_2^0 之间的欧式距离 $D_{1,2}$ 大于阈值 Th_D , 则将 NH_2 放入 Sta 中;

同样方法, 依次将栈外的 $h_i (i = 3, \dots, n')$ 与栈 Sta 内已经存在的哈希路径进行比较, 若满足上述两个条件之一, 将 $NH_i = (x_i^0, h_i)$ 放入栈中; 栈中元素出栈, 并将哈希特征值转化为对应的卷积神经网络模型的输入特征向量;

得到优化后的样本集 N ;

S3.2:将优化后的样本集 N 分为训练集 N_1 和测试集 N_2 ;

S3.3:当新数据不断到来时, 返回步骤S3.2, 对样本集重新进行优化, 模型继续进行学习。

3.根据权利要求1或2所述的卷积神经网络结合遗传算法的智能装备软件测试方法, 其特征在于: 步骤S1中基于变异分支相关图生成路径集被实施为:

设被测程序为 P , 插入 P 的变异分支集合为 $M = \{M_1, M_2, \dots, M_m\}$, $M_1, M_2, \dots, M_m$ 为变异分支, m 为变异分支个数; 由变异分支形成的可执行路径集合设为 $S = \emptyset$;

S1.1:计算变异分支相关度

根据覆盖变异分支的样本数量和总样本数量之间的关系, 可以计算变异分支 M_i 的覆盖难度 $dif(M_i)$;

设变异分支 M_i, M_j 之间的相关度为 $t_{i,j}$; 根据同时覆盖变异分支 M_i, M_j 的样本数量与总样本数量比值率, 估算变异分支之间相关度 $t_{i,j}$; 同样的方法, 可以得到所有的变异分支

$M_1, M_2, \dots, M_m$ 之间的相关度,并构建变异分支相关度矩阵 Λ ;

如果 M_i, M_j 之间的相关度 $t_{i,j} = 0$,则设置 M_i, M_j 为一对冲突变异分支,标记为 $M_i \otimes M_j$;

S1.2:基于变异分支相关度生成变异分支相关图

设变异分支相关图为 $G = \{V, E, W\}$,其中 V 为顶点集合, E 为边的集合, W 为边与边的权重集合,这些集合初值为空集;

首先遍历相关度矩阵 Λ ,当变异分支 M_i, M_j 之间的相关度 $t_{i,j} > 0$ 时,将 M_i, M_j 添加到图 G 的顶点集合 V 中,即 $V = V \cup \{M_i, M_j\}$,在边集合 E 中添加一条由 M_i 指向 M_j 的实边 $\langle M_i, M_j \rangle$,边集合 $E = E \cup \{\langle M_i, M_j \rangle\}$;边 $\langle M_i, M_j \rangle$ 设置权重为 M_i, M_j 之间的相关度 $t_{i,j}$,权重集合 $W = W \cup \{t_{i,j}\}$;根据上面的方法,生成变异分支相关图 $G = \{V, E, W\}$;

S1.3:基于变异分支相关图生成路径集

从变异分支集合 $M = \{M_1, M_2, \dots, M_m\}$ 中,选择覆盖难度最高的变异分支 M_i ,作为构建路径 S_l 的基准节点,并将 M_i 定位在变异分支相关图 G 中。

4.根据权利要求3所述的卷积神经网络结合遗传算法的智能装备软件测试方法,其特征在于:步骤S1.3中基于变异分支相关图生成路径集被实施为:

S1.3.1:设变异分支 M_i 为变异分支相关图 G 中当前节点,在变异分支相关图 G 上沿着 M_i 的后继方向遍历,找出与当前 M_i 相关度最大的后继节点 M_j ,并将 M_j 添加到路径中,此时路径 $S_l = M_i, M_j$;设置 M_j 为当前节点,继续从 M_j 往后继方向查找与 M_j 相关度最大的节点 M_p ,如果节点 M_p 和当前路径中所有结点都不是冲突变异分支,将 M_p 添加到当前路径中,此时 $S_l = M_i, M_j, M_p$;如果 M_p 和当前路径中为某一变异分支节点 M_i 为冲突变异分支,即 $M_p \otimes M_i$,则 M_p 不能添加到路径上,然后,继续寻找与当前节点 M_j 相关度高的下一个节点;

按照上述方法,继续向后继方向遍历,直到所有后继节点都遍历完成,获得路径 $S_l = M_i, M_j, M_p, \dots$,其中 M_i, M_j, M_p 为变异分支;

S1.3.2:设 M_i 为变异分支相关图 G 中当前节点,在变异分支相关图 G 上沿着 M_i 的前驱方向遍历,找出与当前节点 M_i 相关度最大的前驱节点 M_j' ,并将 M_j' 添加到当前路径中,此时 $S_l = M_j', M_i, M_j, M_p, \dots$;随后将 M_j' 作为当前节点,继续往前驱方向查找下一个节点 M_p' ;如果节点 M_p' 和当前路径中所有结点都不是冲突变异分支,将 M_p' 添加到当前路径中, $S_l = M_p', M_j', M_i, M_j, M_p, \dots$;如果节点 M_p' 和当前路径中某一节点为一对冲突变异分支,则 M_p' 为不能添加到路径节点;然后,继续寻找与 M_j' 相关度高的下一个节点;

按照上述方法,继续向前驱方向遍历,直到所有前驱节点都遍历完成,获得路径 $S_l = \dots, M_p', M_j', M_i, M_j, M_p, \dots$;从变异分支集合 M 中删除 S_l 中包含的变异分支;将 S_l 添加到可执行路径集合 S 中,即 $S = S \cup \{S_l\}$;

S1.3.3:从更新过的变异分支集合 M 中,再次选择覆盖难度最高的变异分支,作为路径 S_{l+1} 的基准节点,重复上述S1.3.1到S1.3.2步骤,生成 S_{l+1} ;最终,生成可执行路径集合 $S = \{S_1, S_2, \dots, S_{|S|}\}$, $|S|$ 为路径集中路径条数, $S_1, S_2, \dots, S_{|S|}$ 为可执行路径。

5.根据权利要求2所述的卷积神经网络结合遗传算法的智能装备软件测试方法,其特

征在于:步骤S4增量学习卷积神经网络和多种群遗传方法结合生成测试用例的具体步骤为:

S4.1:设种群为 $G = \{G_1, G_2, \dots, G_q\}$, q 为子种群个数, $G_1, G_2, \dots, G_q$ 为子种群, 子种群 G_l 负责生成覆盖路径 S_l 的测试用例;设置方法所需各种参数的值;

S4.2:利用增量学习卷积神经网络模型,估算高适应度的个体,作为 G_l 的初始种群;

S4.3:判断方法是否满足终止条件,若是,转步骤S4.7;

S4.4:执行被测程序,计算 G_l 中的进化个体的适应度值;

S4.5:比较 G_l 中不同进化个体的性能,进行选择、交叉,变异等遗传操作,产生并保存新的个体;

S4.6:新的个体作为新数据继续输入增量学习卷积神经网络样本集,转到S4.2;

S4.7:停止进化,输出测试用例集。

卷积神经网络结合遗传算法的智能装备软件测试方法

技术领域

[0001] 本发明涉及计算机复杂软件缺陷检测领域,具体是一种卷积神经网络结合遗传算法的智能装备软件测试方法。

背景技术

[0002] 智能装备融合复杂机械结构、精密传感器与先进通信模块,软件是赋予其智能并实现自动化与精准决策的关键。全面严谨的软件测试,能在装备启用前精准定位并修复软件缺陷,防止因软件故障致使智能装备失稳、效率降低,乃至引发安全事故等严重问题。软件测试是保证软件质量的重要手段,通过发现和修复程序中的缺陷,可以有效提高软件的可靠性。其中,变异测试可评估测试用例质量、改进测试过程并提高软件质量。变异测试首先通过选择变异算子,并对程序的某一语句进行合乎语法的微小修改,生成一个新的程序,这个新生成的程序称为变异体。然而,现实中变异测试需要生成大量的变异体。使用同一个测试用例分别执行原程序和变异体,如果两者的输出不同,则测试用例杀死了该变异体,这种准则被称为强变异测试。很显然,对每个变异体执行测试用例,这会消耗大量的计算资源和时间,这使得强变异测试的执行代价非常高,对于大型复杂软件系统来说可能变得难以承受。为了解决这一问题,弱变异测试被提出。

[0003] 最近,越来越多的研究人员将机器学习和深度学习技术集成到软件测试中。卷积神经网络是一种深度学习模型,通过卷积操作从输入数据中提取特征,实现分类和识别等任务。卷积神经网络可以处理大规模数据集,而不会显著增加计算复杂度。此外,卷积神经网络减少了对大量特征工程的需求,增强了数据处理的鲁棒性。

[0004] 遗传方法能够在广阔的搜索空间中进行全局搜索,避免陷入局部最优解。在测试用例生成中,利用选择、交叉和变异等操作不断优化解,这意味着可以找到更广泛的输入组合,以覆盖不同的程序路径和状态,提高测试的全面性。进化方法在测试用例生成中虽表现出色,然而评估每个个体需通过执行被测程序以计算适应度值,随着优化问题复杂性的增加,这一过程会导致执行时间大幅提升,进而增加了测试成本,成为进化方法深入应用的瓶颈。为了解决这一问题,机器学习方法和深度学习中的预测模型被应用。预测模型优化进化方法的过程主要包括数据收集与分析、建立预测模型、引导进化过程以及模型更新与优化等。

发明内容

[0005] 本发明为了解决现有技术中检测复杂智能装备软件众多缺陷的测试用例生成效率低下的问题,提出了一种面向智能装备软件缺陷的卷积神经网络与遗传方法结合的测试方法,该方法区别于原有方法的特色在于,首先,采用变异测试方法,生成代表缺陷的变异分支,再基于变异分支的覆盖难度和变异分支之间相关度生成可执行的路径;其次,然后针对多路径构建多任务测试用例生成数学模型,随后构建基于增量学习的卷积神经网络模型;最后,采用增量学习卷积神经网络增强遗传方法生成具有高缺陷检测能力的测试用例。

[0006] 本发明采用的技术方案如下：

一种卷积神经网络结合遗传算法的智能装备软件测试方法，包括以下步骤：

S1: 基于变异分支相关图生成路径集；

S2: 基于生成的路径集构建测试用例生成优化模型；

S3: 利用所述优化模型按照预设规则构建增量学习卷积神经网络模型；

S4: 将所述增量学习卷积神经网络和多种群遗传方法结合生成测试用例。

[0007] 优选的，所述步骤S3中预设规则被实施为：

S3.1: 优化初始样本集 $N^0 = \{X^0, P(X^0), Fit(X^0)\}$, $X^0 = \{x_1^0, x_2^0, \dots, x_{n'}^0\}$ 为初始的数据集, n' 为初始样本数量; $P(X^0)$ 为基于初始的数据集获得的对应的穿越路径集合; $Fit(X^0)$ 为 X^0 对应的适应值集合;

S3.1.1: 针对初始穿越路径集 $P(X^0)$ 中每一条路径 $p_1(x_1^0), p_2(x_2^0), \dots, p_{n'}(x_{n'}^0)$, 获得哈希编码路径 $h_1, h_2 \dots h_{n'}$, 哈希编码路径集合为 $H = \{h_1, h_2 \dots h_{n'}\}$; 设 $z_i^0 = (x_i^0, p_i(x_i^0))$ 为一个输入特征值, 由它转化为哈希特征值 $NH_i = (x_i^0, h_i)$;

S3.1.2: 将 $NH_1 = (x_1^0, h_1)$ 入栈 Sta 中; 栈外的 $NH_2 = (x_2^0, h_2)$ 是否能入栈 Sta , 满足下面两个条件之一:

(I) 将 h_2 与栈内的 h_1 进行比较, 如果 h_2 和 h_1 哈希编码不相同, 直接将 NH_2 放入 Sta 中;

(II) 如果 h_2 和 h_1 哈希编码相同, 则继续比较 x_1^0 和 x_2^0 , 如果 x_1^0 和 x_2^0 之间的欧式距离 $D_{1,2}$ 大于阈值 Th_D , 则将 NH_2 放入 Sta 中;

同样方法, 依次将栈外的 $h_i (i = 3, \dots, n')$ 与栈 Sta 内已经存在的哈希路径进行比较, 若满足上述两个条件之一, 将 $NH_i = (x_i^0, h_i)$ 放入栈中; 栈中元素出栈, 并将哈希特征值转化为对应的卷积神经网络模型的输入特征向量;

得到优化后的样本集 N ;

S3.2: 将优化后的样本集 N 分为训练集 N_1 和测试集 N_2 ;

S3.3: 当新数据不断到来时, 返回步骤S3.2, 对样本集重新进行优化, 模型继续进行学习。

[0008] 一种卷积神经网络结合遗传算法的智能装备软件测试方法，包括以下步骤：

S1: 基于变异分支相关图生成路径集；

S2: 基于生成的路径集构建测试用例生成优化模型；

S3: 利用所述模型构建增量学习卷积神经网络模型；

S3.1: 优化初始样本集 $N^0 = \{X^0, P(X^0), Fit(X^0)\}$, $X^0 = \{x_1^0, x_2^0, \dots, x_{n'}^0\}$ 为初始的数据集, n' 为初始样本数量; $P(X^0)$ 为基于初始的数据集获得的对应的穿越路径集合; $Fit(X^0)$ 为 X^0 对应的适应值集合;

S3.1.1: 针对初始穿越路径集 $P(X^0)$ 中每一条路径 $p_1(x_1^0), p_2(x_2^0), \dots, p_{n'}(x_{n'}^0)$, 获

得哈希编码路径 $h_1, h_2 \cdots h_{n'}$, 哈希编码路径集合为 $H = \{h_1, h_2 \cdots h_{n'}\}$; 设 $z_i^0 = (x_i^0, p_i(x_i^0))$ 为一个输入特征值, 由它转化为哈希特征值 $NH_i = (x_i^0, h_i)$;

S3.1.2: 将 $NH_1 = (x_1^0, h_1)$ 入栈 Sta 中; 栈外的 $NH_2 = (x_2^0, h_2)$ 是否能入栈 Sta , 满足下面两个条件之一:

(I) 将 h_2 与栈内的 h_1 进行比较, 如果 h_2 和 h_1 哈希编码不相同, 直接将 NH_2 放入 Sta 中;

(II) 如果 h_2 和 h_1 哈希编码相同, 则继续比较 x_1^0 和 x_2^0 , 如果 x_1^0 和 x_2^0 之间的欧式距离 $D_{1,2}$ 大于阈值 Th_D , 则将 NH_2 放入 Sta 中;

同样方法, 依次将栈外的 $h_i (i = 3, \cdots n')$ 与栈 Sta 内已经存在的哈希路径进行比较, 若满足上述两个条件之一, 将 $NH_i = (x_i^0, h_i)$ 放入栈中; 栈中元素出栈, 并将哈希特征值转化为对应的卷积神经网络模型的输入特征向量;

得到优化后的样本集 N ;

S3.2: 将优化后的样本集 N 分为训练集 N_1 和测试集 N_2 ;

S3.3: 当新数据不断到来时, 返回步骤S3.2, 对样本集重新进行优化, 模型继续进行学习;

S4: 将所述增量学习卷积神经网络和多种群遗传方法结合生成测试用例。

[0009] 另外还包括: 一种卷积神经网络结合遗传算法的智能装备软件测试方法, 该方法包含以下步骤:

[0010] S1: 基于变异分支相关图生成路径集

[0011] 设被测程序为 P , 插入 P 的变异分支集合为 $M = \{M_1, M_2, \cdots, M_m\}$, $M_1, M_2, \cdots, M_m$ 为变异分支, m 为变异分支个数; 由变异分支形成的可执行路径集合设为 $S = \emptyset$;

[0012] S1.1: 计算变异分支相关度

[0013] 根据覆盖变异分支的样本数量和总样本数量之间的关系, 可以计算变异分支 M_i 的覆盖难度 $dif(M_i)$;

[0014] 设变异分支 M_i, M_j 之间的相关度为 $t_{i,j}$; 根据同时覆盖变异分支 M_i, M_j 的样本数量与总样本数量比值率, 估算变异分支之间相关度 $t_{i,j}$; 同样的方法, 可以得到所有的变异分支 $M_1, M_2, \cdots, M_m$ 之间的相关度, 并构建变异分支相关度矩阵 A ;

[0015] 如果 M_i, M_j 之间的相关度 $t_{i,j} = 0$, 则设置 M_i, M_j 为一对冲突变异分支, 标记为 $M_i \otimes M_j$ 。

[0016] S1.2: 基于变异分支相关度生成变异分支相关图

[0017] 设变异分支相关图为 $G = \{V, E, W\}$, 其中 V 为顶点集合, E 为边的集合, W 为边与边的权重集合, 这些集合初值为空集;

[0018] 首先遍历相关度矩阵 A , 当变异分支 M_i, M_j 之间的相关度 $t_{i,j} > 0$ 时, 将 M_i, M_j 添加到图 G 的顶点集合 V 中, 即 $V = V \cup \{M_i, M_j\}$, 在边集合 E 中添加一条由 M_i 指向 M_j 的实边 $\langle M_i, M_j \rangle$, 集合 $E = E \cup \{\langle M_i, M_j \rangle\}$ 。边 $\langle M_i, M_j \rangle$ 设置权重为 M_i, M_j 之间的相关度 $t_{i,j}$, 权重集合

$W = W \cup \{t_{i,j}\}$;根据上面的方法,生成变异分支相关图 $G = \{V, E, W\}$ 。

[0019] S1.3:基于变异分支相关图生成路径集

[0020] 从变异分支集合 $M = \{M_1, M_2, \dots, M_m\}$ 中,选择覆盖难度最高的变异分支 M_i ,作为构建路径 S_l 的基准节点,并将 M_i 定位在变异分支相关图 G 中;

[0021] S1.3.1:设变异分支 M_i 为图 G 中当前节点,在图 G 上沿着 M_i 的后继方向遍历,找出与当前 M_i 相关度最大的后继节点 M_j ,并将 M_j 添加到路径中,此时路径 $S_l = M_i, M_j$;设置 M_j 为当前节点,继续从 M_j 往后继方向查找与 M_j 相关度最大的节点 M_p ,如果节点 M_p 和当前路径中所有结点都不是冲突变异分支,将 M_p 添加到当前路径中,此时 $S_l = M_i, M_j, M_p$;如果 M_p 和当前路径中为某一变异分支节点 M_i 为冲突变异分支,即 $M_p \otimes M_i$,则 M_p 不能添加到路径上,然后,继续寻找与当前节点 M_j 相关度高的下一个节点;

[0022] 按照上面的方法,继续向后继方向遍历,直到所有后继节点都遍历完成,获得路径 $S_l = M_i, M_j, M_p, \dots$,其中 M_i, M_j, M_p 为变异分支;同时,从变异分支集合 M 中删除 S_l 上包含的变异分支;

[0023] S1.3.2:设 M_i 为图 G 中当前节点,在图 G 上沿着 M_i 的前驱方向遍历,找出与当前节点 M_i 相关度最大的前驱节点 M_j' ,并将 M_j' 添加到当前路径中,此时 $S_l = M_j', M_i, M_j, M_p, \dots$;随后将 M_j' 作为当前节点,继续往前驱方向查找下一个节点 M_p' ;如果节点 M_p' 和当前路径中所有结点都不是冲突变异分支,将 M_p' 添加到当前路径中, $S_l = M_p', M_j', M_i, M_j, M_p, \dots$;如果节点 M_p' 和当前路径中某一节点为一对冲突变异分支,则 M_p' 为不能添加到路径节点;然后,继续寻找与 M_j' 相关度高的下一个节点;

[0024] 按照上面的方法,继续向前驱方向遍历,直到所有前驱节点都遍历完成,获得路径 $S_l = \dots, M_p', M_j', M_i, M_j, M_p, \dots$;从变异分支集合 M 中删除 S_l 中包含的变异分支;将 S_l 添加到可执行路径集合 S 中,即 $S = S \cup \{S_l\}$;

[0025] S1.3.3:从更新过的变异分支集合 M 中,再次选择覆盖难度最高的变异分支,作为路径 S_{l+1} 的基准节点,重复上述S1.3.1到S1.3.2步骤,生成 S_{l+1} ;最终,生成可执行路径集合 $S = \{S_1, S_2, \dots, S_{|S|}\}$, $|S|$ 为路径集中路径条数, $S_1, S_2, \dots, S_{|S|}$ 为可执行路径。

[0026] S2:基于生成的路径集构建测试用例生成优化模型;

[0027] 针对每一条目标路径 S_l ,建立基于路径覆盖的测试用例生成优化模型,可以表示为:

$$\begin{aligned} & \max(f_l(X)) \\ & \text{s.t. } X \in D(X) \end{aligned}$$

[0029] 其中 $D(X)$ 为 X 所形成的取值域; $f_l(X)$ 为目标函数,它可以被定义为路径之前的相似度;

$$f_l(X) = \frac{|P(X) \Delta S_l|}{\max(|S_l|, |P(X)|)}$$

[0031] 其中, $|S_l|$ 为目标路径包含的节点个数, $P(X)$ 为测试用例 X 穿越路径包含的节点个数; $|P(X) \Delta S_l|$ 为穿越路径 $P(X)$ 和目标路径 S_l 之间连续相同的节点数;

[0032] S3:利用生成的优化模型构建增量学习卷积神经网络模型;

[0033] S3.1:利用生成的优化模型构建神经网络模型的初始样本集 $N^0 = \{X^0, P(X^0), Fit(X^0)\}$, 式中, $X^0 = \{x_1^0, x_2^0, \dots, x_n^0\}$ 为初始的数据集, n' 为初始样本数量, $x_1^0, x_2^0, \dots, x_n^0$ 为初始数据; $P(X^0) = \{p_1(x_1^0), p_2(x_2^0), \dots, p_{n'}(x_n^0)\}$ 为基于初始的数据集获得的对应的穿越路径集合; $Fit(X^0)$ 为 $X^0 = \{x_1^0, x_2^0, \dots, x_n^0\}$ 对应的适应值集合; 定义增量学习卷积神经网络的卷积层, 池化层以及输出层; 指定损失函数为交叉熵损失, 设定评估模型性能的指标为准确度;

[0034] 增量学习卷积神经网络初始化:

[0035] 定义增量学习卷积神经网络的卷积层, 池化层以及输出层; 设模型的输入特征为 X^0 , $P(X^0)$ 为辅助特征; 输出特征为 $Fit(X^0)$;

[0036] 编译模型:

[0037] 指定损失函数为交叉熵损失, 设定评估模型性能的指标为准确度;

[0038] S3.2: 优化初始样本集 N^0

[0039] S3.2.1: 针对初始穿越路径集 $P(X^0)$ 中每一条路径 $p_1(x_1^0), p_2(x_2^0), \dots, p_{n'}(x_n^0)$, 采用哈希函数, 获得哈希编码路径 $h_1, h_2 \dots h_{n'}$, 哈希编码路径集合为 $H = \{h_1, h_2 \dots h_{n'}\}$; 设 $z_i^0 = (x_i^0, p_i(x_i^0))$ 为一个输入特征值, 由它转化为哈希特征值 $NH_i = (x_i^0, h_i)$, 由此得到哈希特征集合 $NH = \{NH_1, NH_2, \dots, NH_{n'}\}$;

[0040] S3.2.2: 初始化栈 Sta , 将哈希特征值 $NH_1 = (x_1^0, h_1^0)$ 入栈 Sta 中; 栈外的 $NH_2 = (x_2^0, h_2^0)$ 是否能入栈 Sta , 需要满足下面两个条件之一:

[0041] (I) h_2 与栈内的 h_1 进行比较, 如果 h_2 和 h_1 哈希编码不相同, 直接将 NH_2 放入 Sta 中;

[0042] (II) 如果 h_2 和 h_1 哈希编码相同, 那么继续比较 x_1^0 和 x_2^0 , 如果 x_1^0 和 x_2^0 之间的欧式距离 $D_{1,2}$ 大于阈值 Th_D , 那么, 将 NH_2 放入 Sta 中;

[0043] 随后, 按照同样的方法, 依次将栈外的 $h_i (i = 3, \dots, n)$ 与栈 Sta 内已经存在的哈希路径进行比较, 如果满足上面两个条件之一, 则将 $NH_i = (x_i^0, h_i)$ 放入栈中; 最后, 栈中元素出栈, 并将哈希特征值转化为对应的卷积神经网络模型的输入特征向量;

[0044] 最后得到优化后的样本集 $N = \{(z_1, fit(x_1)), (z_2, fit(x_2)), \dots, (z_n, fit(x_n))\}$, 其中 n 为优化后样本数量, $z_i = (x_i, p_i(x_i))$ 为优化后的输入特征值, $fit(x_1), fit(x_2), \dots, fit(x_n)$ 为 $x_1, x_2, \dots, x_n$ 对应的适应值;

[0045] S3.3: 将优化后的样本集 N 分为训练集 N_1 和测试集 N_2 ;

[0046] 训练集 N_1 用于训练神经网络模型, 设模型的输入特征为 X , 辅助特征为 $P(X)$; 输出特征为 $Fit(X)$;

[0047] S3.4: 当新数据不断到来时, 返回步骤 S3.2, 对样本集重新进行优化, 模型继续进行学习, 不需要重新训练整个模型。

[0048] 图2为本发明所构建的基于增量学习卷积神经网络模型框架图, 其中内层框图为

本发明改进传统卷积神经网络,优化样本集和添加辅助特征 $P(X)$;外层框图为不断更新模型,形成增量学习卷积神经网络模型。

[0049] S4:将所述增量学习卷积神经网络和多种群遗传方法结合生成测试用例。

[0050] 具体步骤为:

[0051] S4.1:设种群为 $G = \{G_1, G_2, \dots, G_q\}$, q 为子种群个数,子种群 G_l 负责生成覆盖路径 S_l 的测试用例;设置方法所需各种参数的值;

[0052] S4.2:利用增量学习卷积神经网络模型,估算高适应度的个体,作为 G_l 的初始种群;

[0053] S4.3:判断方法是否满足终止条件,若是,转S4.7;

[0054] S4.4:执行被测程序,计算 G_l 中的进化个体的适应度值;

[0055] S4.5:比较 G_l 中不同进化个体的性能,进行选择、交叉、变异等遗传操作,产生并保存新的个体;

[0056] S4.6:新的个体作为新数据继续输入增量学习卷积神经网络样本集,转到S4.2;

[0057] S4.7:停止进化,输出测试用例集。

[0058] 上面S4.3中终止条件有两个,一个是生成期望的测试用例;另一个是种群进化到最大迭代次数;

[0059] 所述S4.4中适应值由函数 $fit_l(X)$ 由式(2)的目标函数确定:

[0060] $fit_l(X) = f_l(X)$

[0061] 本发明的有益效果:

[0062] (1)利用图论方法双向遍历变异分支相关图生成路径,此方法确保生成的路径集保持最小,同时实现程序中变异分支的覆盖。通过对应一个紧凑的测试用例集,大大减少了测试过程中的时间和资源消耗,从而提高了变异测试的效率。可以在不增加硬件成本的前提下,实现对软件系统的全面、高效测试,确保软件在有限资源下的可靠性和稳定性。

[0063] (2)本发明提出了增量学习卷积神经网络,增加了一个辅助输入特征,以及哈希函数来优化样本集。此外,通过采用增量学习方法,在测试用例生成过程中实时动态更新训练集。这种增量学习优化提高了模型的准确性和有效性,确保测试用例集在整个过程中不断改进。

[0064] (3)增量学习卷积神经网络和多种群遗传方法结合生成测试用例时,增量学习卷积神经网络可以快速评估进化个体的适应度值,从而提高了多种群遗传方法的进化效率。此外,增量学习卷积神经网络支持的实时更新允许持续改进,随着时间的推移,获得更高质量的测试用例。该方法通过子种群并行进化,可以有效地生成多路径的测试用例。该方法还可以有效地扩展日益复杂的软件系统,为智能装备软件实施大规模测试提供了一个健壮的解决方案。

附图说明

[0065] 图1为本发明一种卷积神经网络结合遗传算法的智能装备软件测试方法的总流程图;

[0066] 图2为本发明所提增量学习卷积神经网络框架图;

- [0067] 图3为本发明实施示例程序；
 [0068] 图4为示例程序包含变异分支的覆盖难度的雷达图；
 [0069] 图5为示例程序变异分支之前相关度矩阵的热力图；
 [0070] 图6为示例程序变异分支相关图；
 [0071] 图7为样本集优化过程图

具体实施方式

[0072] 图1为本发明一个实施例提供的一种卷积神经网络结合遗传算法的智能装备软件测试方法总流程图；在本实施例中，卷积神经网络结合遗传算法的智能装备软件测试方法包括如下步骤：

[0073] S1:基于变异分支相关图生成路径集

[0074] 设被测程序为 P ，插入 P 的变异分支集合为 $M = \{M_1, M_2, \dots, M_m\}$ ， $M_1, M_2, \dots, M_m$ 为变异分支， m 为变异分支个数；由变异分支形成的可执行路径集合设为 $S = \emptyset$ ；

[0075] S1.1:计算变异分支相关度

[0076] 根据覆盖变异分支的样本数量和总样本数量之间的关系，可以计算变异分支 M_i 的覆盖难度 $dif(M_i)$ ；

[0077] 设变异分支 M_i, M_j 之间的相关度为 $t_{i,j}$ ；根据同时覆盖变异分支 M_i, M_j 的样本数量与总样本数量比值率，估算变异分支之间相关度 $t_{i,j}$ ；同样的方法，可以得到所有的变异分支 $M_1, M_2, \dots, M_m$ 之间的相关度，并构建变异分支相关度矩阵 A ；

[0078] 如果 M_i, M_j 之间的相关度 $t_{i,j} = 0$ ，则设置 M_i, M_j 为一对冲突变异分支，标记为 $M_i \otimes M_j$ 。

[0079] S1.2:基于变异分支相关度生成变异分支相关图

[0080] 设变异分支相关图为 $G = \{V, E, W\}$ ，其中 V 为顶点集合， E 为边的集合， W 为边与边的权重集合，这些集合初值为空集；

[0081] 首先遍历相关度矩阵 A ，当变异分支 M_i, M_j 之间的相关度 $t_{i,j} > 0$ 时，将 M_i, M_j 添加到图 G 的顶点集合 V 中，即 $V = V \cup \{M_i, M_j\}$ ，在边集合 E 中添加一条由 M_i 指向 M_j 的实边 $\langle M_i, M_j \rangle$ ，集合 $E = E \cup \{\langle M_i, M_j \rangle\}$ 。边 $\langle M_i, M_j \rangle$ 设置权重为 M_i, M_j 之间的相关度 $t_{i,j}$ ，权重集合 $W = W \cup \{t_{i,j}\}$ ；根据上面的方法，生成变异分支相关图 $G = \{V, E, W\}$ 。

[0082] S1.3:基于变异分支相关图生成路径集

[0083] 从变异分支集合 $M = \{M_1, M_2, \dots, M_m\}$ 中，选择覆盖难度最高的变异分支 M_i ，作为构建路径 S_i 的基准节点，并将 M_i 定位在变异分支相关图 G 中；

[0084] S1.3.1:设变异分支 M_i 为图 G 中当前节点，在图 G 上沿着 M_i 的后继方向遍历，找出与当前 M_i 相关度最大的后继节点 M_j ，并将 M_j 添加到路径中，此时路径 $S_i = M_i, M_j$ ；设置 M_j 为当前节点，继续从 M_j 往后继方向查找与 M_j 相关度最大的节点 M_p ，如果节点 M_p 和当前路径中所有结点都不是冲突变异分支，将 M_p 添加到当前路径中，此时 $S_i = M_i, M_j, M_p$ ；如果 M_p 和当前路径中为某一变异分支节点 M_i 为冲突变异分支，即 $M_p \otimes M_i$ ，则 M_p 不能添加到路径

上,然后,继续寻找与当前节点 M_j 相关度高的下一个节点;

[0085] 按照上面的方法,继续向后继方向遍历,直到所有后继节点都遍历完成,获得路径 $S_l = M_i, M_j, M_p, \dots$,其中 M_i, M_j, M_p 为变异分支;同时,从变异分支集合 M 中删除 S_l 上包含的变异分支;

[0086] S1.3.2:设 M_i 为图 G 中当前节点,在图 G 上沿着 M_i 的前驱方向遍历,找出与当前节点 M_i 相关度最大的前驱节点 M_j' ,并将 M_j' 添加到当前路径中,此时 $S_l = M_j', M_i, M_j, M_p, \dots$;随后将 M_j' 作为当前节点,继续往前驱方向查找下一个节点 M_p' ;如果节点 M_p' 和当前路径中所有结点都不是冲突变异分支,将 M_p' 添加到当前路径中, $S_l = M_p', M_j', M_i, M_j, M_p, \dots$;如果节点 M_p' 和当前路径中某一节点为一对冲突变异分支,则 M_p' 为不能添加到路径节点;然后,继续寻找与 M_j' 相关度高的下一个节点;

[0087] 按照上面的方法,继续向前驱方向遍历,直到所有前驱节点都遍历完成,获得路径 $S_l = \dots, M_p', M_j', M_i, M_j, M_p, \dots$;从变异分支集合 M 中删除 S_l 中包含的变异分支;将 S_l 添加到可执行路径集合 S 中,即 $S = S \cup \{S_l\}$;

[0088] S1.3.3:从更新过的变异分支集合 M 中,再次选择覆盖难度最高的变异分支,作为路径 S_{l+1} 的基准节点,重复上述S1.3.1到S1.3.2步骤,生成 S_{l+1} ;最终,生成可执行路径集合 $S = \{S_1, S_2, \dots, S_{|S|}\}$, $|S|$ 为路径集中路径条数, $S_1, S_2, \dots, S_{|S|}$ 为可执行路径。

[0089] S2:基于生成的路径集构建测试用例生成优化模型;

[0090] 针对每一条目标路径 S_l ,建立基于路径覆盖的测试用例生成优化模型,可以表示为:

$$\begin{aligned} & \max(f_l(X)) \\ & \text{s.t. } X \in D(X) \end{aligned} \quad (1)$$

[0092] 其中 $D(X)$ 为 X 所形成的取值域; $f_l(X)$ 为目标函数,它可以被定义为路径之前的相似度;

$$f_l(X) = \frac{|P(X) \Delta S_l|}{\max(|S_l|, |P(X)|)} \quad (2)$$

[0094] 其中, $|S_l|$ 为目标路径包含的节点个数, $P(X)$ 为测试用例 X 穿越路径包含的节点个数; $|P(X) \Delta S_l|$ 为穿越路径 $P(X)$ 和目标路径 S_l 之间连续相同的节点数;

[0095] S3:利用生成的优化模型构建增量学习卷积神经网络模型;

[0096] S3.1:利用生成的优化模型构建神经网络模型的初始样本集 $N^0 = \{X^0, P(X^0), Fit(X^0)\}$,式中, $X^0 = \{x_1^0, x_2^0, \dots, x_n^0\}$ 为初始的数据集, n' 为初始样本数量, $x_1^0, x_2^0, \dots, x_n^0$ 为初始数据; $P(X^0) = \{p_1(x_1^0), p_2(x_2^0), \dots, p_{n'}(x_n^0)\}$ 为基于初始的数据集获得的对应的穿越路径集合; $Fit(X^0)$ 为 $X^0 = \{x_1^0, x_2^0, \dots, x_n^0\}$ 对应的适应值集合;定义增量学习卷积神经网络的卷积层,池化层以及输出层;指定损失函数为交叉熵损失,设定评估模型性能的指标为准确度;

[0097] 增量学习卷积神经网络初始化:

[0098] 定义增量学习卷积神经网络的卷积层,池化层以及输出层;设模型的输入特征为 X^0 , $P(X^0)$ 为辅助特征;输出特征为 $Fit(X^0)$;

[0099] 编译模型:

[0100] 指定损失函数为交叉熵损失,设定评估模型性能的指标为准确度;

[0101] S3.2:优化初始样本集 N^0

[0102] S3.2.1:针对初始穿越路径集 $P(X^0)$ 中每一条路径 $p_1(x_1^0), p_2(x_2^0), \dots, p_n(x_n^0)$, 采用哈希函数, 获得哈希编码路径 $h_1, h_2 \dots h_n$, 哈希编码路径集合为 $H = \{h_1, h_2 \dots h_n\}$; 设 $z_i^0 = (x_i^0, p_i(x_i^0))$ 为一个输入特征值, 由它转化为哈希特征值 $NH_i = (x_i^0, h_i)$, 由此得到哈希特征集合 $NH = \{NH_1, NH_2, \dots, NH_n\}$;

[0103] S3.2.2:初始化栈 Sta , 将哈希特征值 $NH_1 = (x_1^0, h_1^0)$ 入栈 Sta 中; 栈外的 $NH_2 = (x_2^0, h_2^0)$ 是否能入栈 Sta , 需要满足下面两个条件之一:

[0104] (I) h_2 与栈内的 h_1 进行比较, 如果 h_2 和 h_1 哈希编码不相同, 直接将 NH_2 放入 Sta 中;

[0105] (II) 如果 h_2 和 h_1 哈希编码相同, 那么继续比较 x_1^0 和 x_2^0 , 如果 x_1^0 和 x_2^0 之间的欧式距离 $D_{1,2}$ 大于阈值 Th_D , 那么, 将 NH_2 放入 Sta 中;

[0106] 随后, 按照同样的方法, 依次将栈外的 $h_i (i = 3, \dots, n)$ 与栈 Sta 内已经存在的哈希路径进行比较, 如果满足上面两个条件之一, 则将 $NH_i = (x_i^0, h_i)$ 放入栈中; 最后, 栈中元素出栈, 并将哈希特征值转化为对应的卷积神经网络模型的输入特征向量;

[0107] 最后得到优化后的样本集 $N = \{(z_1, fit(x_1)), (z_2, fit(x_2)), \dots, (z_n, fit(x_n))\}$, 其中 n 为优化后样本数量, $z_i = (x_i, p_i(x_i))$ 为优化后的输入特征值, $fit(x_1), fit(x_2), \dots, fit(x_n)$ 为 $x_1, x_2, \dots, x_n$ 对应的适应值;

[0108] S3.3:将优化后的样本集 N 分为训练集 N_1 和测试集 N_2 ;

[0109] 训练集 N_1 用于训练神经网络模型, 设模型的输入特征为 X , 辅助特征为 $P(X)$; 输出特征为 $Fit(X)$;

[0110] S3.4:当新数据不断到来时, 返回步骤S3.2, 对样本集重新进行优化, 模型继续进行学习, 不需要重新训练整个模型。

[0111] 图2为本发明所构建的基于增量学习卷积神经网络模型框架图, 其中内层框图为本发明改进传统卷积神经网络, 优化样本集和添加辅助特征 $P(X)$; 外层框图为不断更新模型, 形成增量学习卷积神经网络模型。

[0112] S4:将所述增量学习卷积神经网络和多种群遗传方法结合生成测试用例。

[0113] 具体步骤为:

[0114] S4.1:设种群为 $G = \{G_1, G_2, \dots, G_q\}$, q 为子种群个数, 子种群 G_l 负责生成覆盖路径 S_l 的测试用例; 设置方法所需各种参数的值;

[0115] S4.2:利用增量学习卷积神经网络模型, 估算高适应度的个体, 作为 G_l 的初始种群;

[0116] S4.3:判断方法是否满足终止条件, 若是, 转S4.7;

[0117] S4.4: 执行被测程序, 计算 G_l 中的进化个体的适应度值;

[0118] S4.5: 比较 G_l 中不同进化个体的性能, 进行选择、交叉、变异等遗传操作, 产生并保存新的个体;

[0119] S4.6: 新的个体作为新数据继续输入增量学习卷积神经网络样本集, 转到S4.2;

[0120] S4.7: 停止进化, 输出测试用例集。

[0121] 上面S4.3中终止条件有两个, 一个是生成期望的测试用例; 另一个是种群进化到最大迭代次数;

[0122] 所述S4.4中适应值由函数 $fit_l(X)$ 由式 (2) 的目标函数确定:

$$[0123] \quad fit_l(X) = f_l(X) \quad (3)$$

[0124] 图3为本发明的示例程序。图3的 (a) 为程序的源语句, 图3的 (b) 为一个变异体, 图3的 (c) 为插入变异分支的新被测程序。

[0125] 下面通过示例程序说明本发明的实施过程

[0126] 设测试用例集合中, 共有4000个样本总量, 其中有1962个可以覆盖变异分支 M_1 , 那么 M_1 的覆盖难度为 $1 - \frac{1962}{4000} = 0.5095$ 。同理, 可以计算出所有变异分支的覆盖难度, 如图4为示例程序中所有变异分支的覆盖难度的雷达图, 离中心越远的数据点, 表示该指标的数值越大。

[0127] 进一步的, 根据共同杀死变异分支的测试用例数量, 可以估算出变异分支之间的相关度, 从而获得变异分支相关度矩阵, 如图5所示为示例程序变异分支相关度矩阵的热力图;

[0128] 接着, 利用S1.2所述方法, 构建变异分支相关图, 如图6所示。

[0129] 接下来, 基于变异分支相关图生成路径集 S 。首先, 在变异分支集合中, 选择最难覆盖的变异分支 M_5 ($dif(M_5) = 0.9923$)。使用 M_5 作为路径的基准节点, 并将 M_5 定位在图 G 中, 分别在前驱和后继两个方向上遍历变异分支相关图。

[0130] 当 M_5 沿后继方向遍历时, 对应遍历变异分支相关度矩阵的第5行, M_5 相关的后继节点包括 $M_6, M_8, M_9, M_{10}, M_{11}, M_{12}, M_{13}, M_{17}, M_{18}, M_{19}$ 和 M_{20} 。其中, M_5 和 M_6 ($t_{5,6} = 1$) 的相关度最高。因此, 将 M_6 添加到路径中, 即 $S = M_5, M_6$ 。

[0131] 接着, 遍历 M_5 的前驱方向, 即遍历变异分支相关图, 在与 M_5 相关的前驱节点中, M_4 和 M_5 ($t_{4,5} = 0.05$) 的相关度最高。因此, 更新路径 $S = M_4, M_5, M_6$ 。接下来, 重复这个过程, 从 M_4 向前遍历, 从 M_6 向后遍历。

[0132] 需要注意的是, 从 M_{13} 向后遍历时, 当前路径

$S_1 = M_1, M_2, M_3, M_4, M_5, M_6, M_8, M_9, M_{10}, M_{11}, M_{12}, M_{13}$ 。虽然 M_{13} 和 M_{14} 之间的相关度最高, 但 M_{14} 和 M_3 构成一对冲突变异分支 ($M_3 \otimes M_{14}$)。所以不将 M_{14} 添加到路径中。

[0133] 最后, 基于基准节点 M_5 , 遍历变异分支相关图并生成路径 S_1 , $S_1 = M_1, M_2, M_3, M_4, M_5, M_6, M_8, M_9, M_{10}, M_{11}, M_{12}, M_{13}, M_{17}, M_{18}, M_{19}, M_{20}$, 如图6所示的深色线连接的变异分支。

[0134] 接下来, 从变异分支集合中移除路径 S_1 中包含的变异分支。然后从剩余的变异分

支中选择覆盖难度最高的节点 M_7 ,并按照同样的方法生成路径 S_2 ,直到初始的变异分支集合中没有剩余的变异分支为止,其中,

$$S_2 =$$

$M_1, M_2, M_3, M_7, M_8, M_9, M_{10}, M_{11}, M_{12}, M_{13}, M_{14}, M_{15}, M_{16}, M_{17}, M_{18}, M_{19}, M_{20}$

[0135] 最后,可以得到路径集合 $S = \{S_1, S_2\}$ 。

[0136] 那么,基于这两条路径建立覆盖它们的多任务优化模型如下:

$$s. t. X \in D(X)$$

[0137] $T^1: \max(f_1(X)) T^2: \max(f_2(X))$ (4)

$$s. t. X \in D(X)$$

[0138] 然后,为两条路径分别需要构建卷积神经网络。

[0139] 如图7所示,下面举例阐述对样本集进行优化的具体过程。

[0140] 假设初始数据集的输入特征集为 X^0 ,其中包含:

[0141] $x_1^0 = (5,5,5), x_2^0 = (65,65,65), x_3^0 = (1,2,3), x_4^0 = (4,5,6);$

[0142] 然后,通过执行被测程序,得到与初始数据集相对应的穿越路径集 $P(X^0)$,其中包含:

[0143] $P(x_1^0) = 1,2,5,7, P(x_2^0) = 1,2,5,7, P(x_3^0) = 1,2,6,7, P(x_4^0) = 1,2,6,7;$

[0144] 当目标路径 $S_l = \{1,2,4,6,7\}$ 时,可以获得初始数据集 X^0 对应的适应度值集合 $Fit(X^0)$,其中包含:

[0145] $fit(x_1^0) = 0.4, fit(x_2^0) = 0.4, fit(x_3^0) = 0.4, fit(x_4^0) = 0.4;$

[0146] 接着,对穿越路径集合中的每一条路径进行哈希编码,获得哈希编码路径集合 H ,其中包含:

[0147] $h_1 = c3fcd3d76192e4007dfb496cca67e13b$
 $h_2 = c3fcd3d76192e4007dfb496cca67e13b$
 $h_3 = 709b55da7c384b259d728ec6d96adfb3$
 $h_4 = 709b55da7c384b259d728ec6d96adfb3;$

[0148] 由此,得到哈希特征集合 $NH = \{NH_1, NH_2, NH_3, NH_4\}$ 。

[0149] 进一步构建和初始化 Sta 后,将哈希特征值 $NH_1 = (x_1^0, h_1)$ 入栈,接下来的哈希特征值是否进栈时,存在以下三种情况:

[0150] 情况 1:在判断 NH_2 是否要进栈时,首先将 NH_2 与栈内已经存在的哈希特征值 NH_1 进行路径哈希编码比较,发现 $h_2 = h_1$ 。那么就需要继续比较初始数据在搜索空间中的欧氏距离 $D_{1,2} = \sqrt{(65-5)^2 + (65-5)^2 + (65-5)^2} \approx 103.923$

[0151] 由于设定阈值 $Th_D = 6$,而 $D_{1,2}$ 大于 Th_D ($103.923 > 6$),因此 $NH_2 = (x_2, h_2)$ 可以进入 Sta 。

[0152] 情况2:在确定是否应将 NH_3 入栈时,再次进行路径哈希编码比较。结果发现 h_3 与栈中已经存在的 h_2 和 h_1 都不同。因此, NH_3 入栈。

[0153] 情况3:在决定是否将 NH_4 入栈时,发现 h_4 与堆栈中已经存在的 h_3 相同。通过计算

x_4^0 和 x_3^0 在搜索空间中的欧氏距离 $D_{4,3}$,即: $\sqrt{(4-1)^2 + (5-2)^2 + (6-3)^2} \approx 5.196$

[0154] 由于这个距离小于阈值 6, NH_4 不能入栈。

[0155] 随后,从栈中哈希特征值 NH_3, NH_2, NH_1 被依次从栈 Sta 中取出,得到优化后的数据集 $X = \{x_1, x_2, x_3\}$ 。 x_1 和 $P_1(x_1)$ 构成卷积神经网络的输入特征向量, $fit_1(x_1)$ 作为输出,最终得到优化后的样本集 $N = \{(z_1, fit_1(x_1)), (z_2, fit_2(x_2)), (z_3, fit_3(x_3))\}$ 。

[0156] 随后,将 N 分为训练集 N_1 和测试集 N_2 ,并使用 N_1 训练卷积神经网络模型, N_2 评估卷积神经网络模型的准确度。当进化方法生成新个体时,当作新数据返回步骤S3.2,对样本集重新进行优化,模型继续进行学习。

[0157] 采用多种群遗传方法生成测试用例时,首先使用卷积神经网络模型生成适应度高的个体,作为多种群遗传方法的初始种群。

[0158] 多种群遗传方法设置子种群个数为3,子种群规模为5。多种群遗传方法的进化代数设为4000。遗传操作采用轮盘赌选择,单点交叉和单点变异,交叉概率和变异概率分别为0.9和0.3。最后,得到杀死变异分支的测试用例集合为 $\{(46,56,46), (8,7,6)\}$ 。

[0159] 需要说明的是:上述本发明实施例先后顺序仅仅为了描述,不代表实施例的优劣。且上述对本说明书特定实施例进行了描述。其它实施例在所附权利要求书的范围内。在一些情况下,在权利要求书中记载的动作或步骤可以按照不同于实施例中的顺序来执行并且仍然可以实现期望的结果。另外,在附图中描绘的过程不一定要求示出的特定顺序或者连续顺序才能实现期望的结果。在某些实施方式中,多任务处理和并行处理也是可以的或者可能是有利的。

[0160] 以上描述是为了进行图示说明而不是为了进行限制。通过阅读上述描述,在所提供的示例之外的许多实施方式和许多应用对本领域技术人员来说都将是显而易见的。因此,本教导的范围不应该参照上述描述来确定,而是应该参照所附权利要求以及这些权利要求所拥有的等价物的全部范围来确定。出于全面之目的,所有文章和参考包括专利申请和公告的公开都通过参考结合在本文中。在前述权利要求中省略这里公开的主题的任何方面并不是为了放弃该主体内容,也不应该认为发明人没有将该主题考虑为所公开的发明主题的一部分。

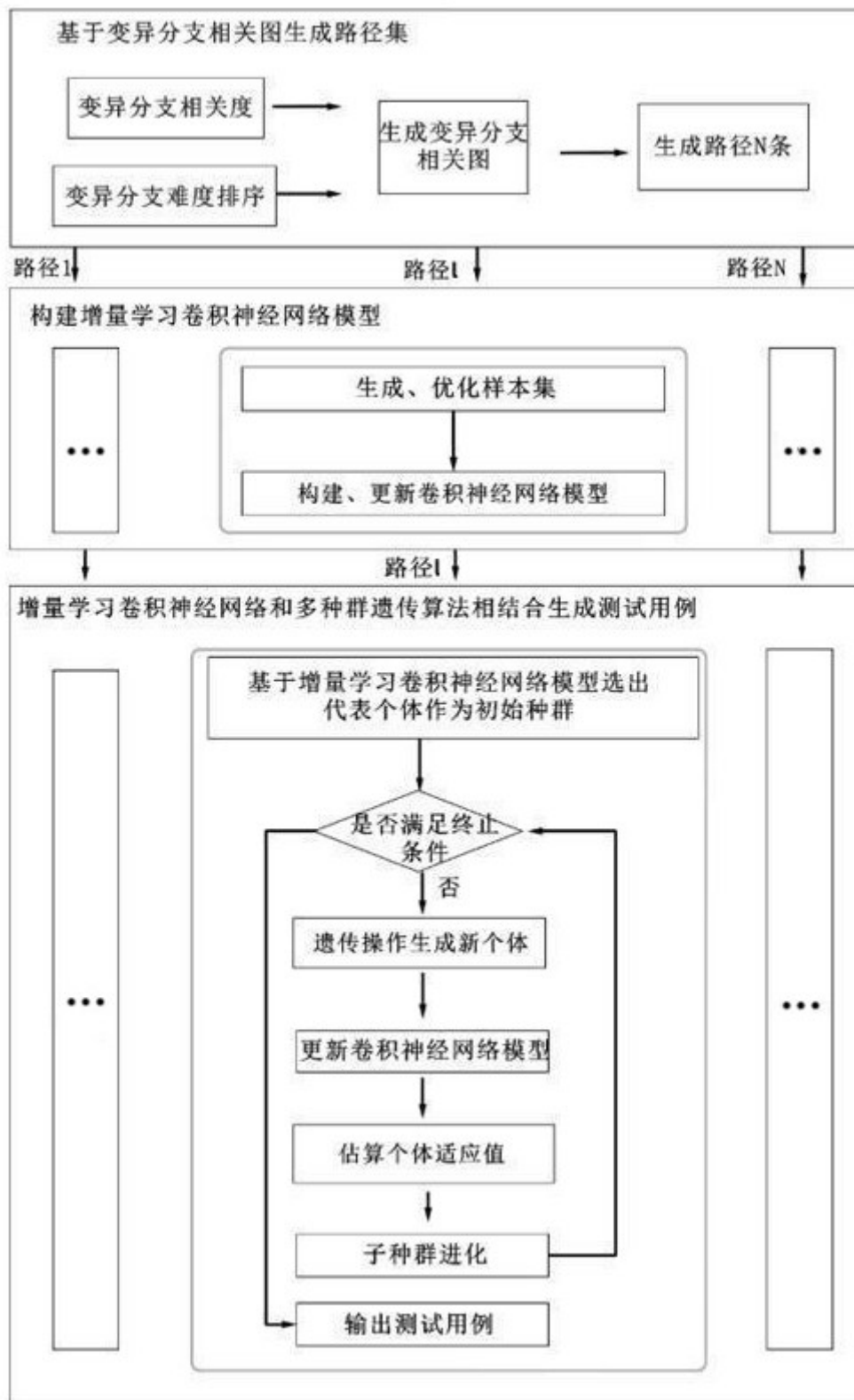


图 1

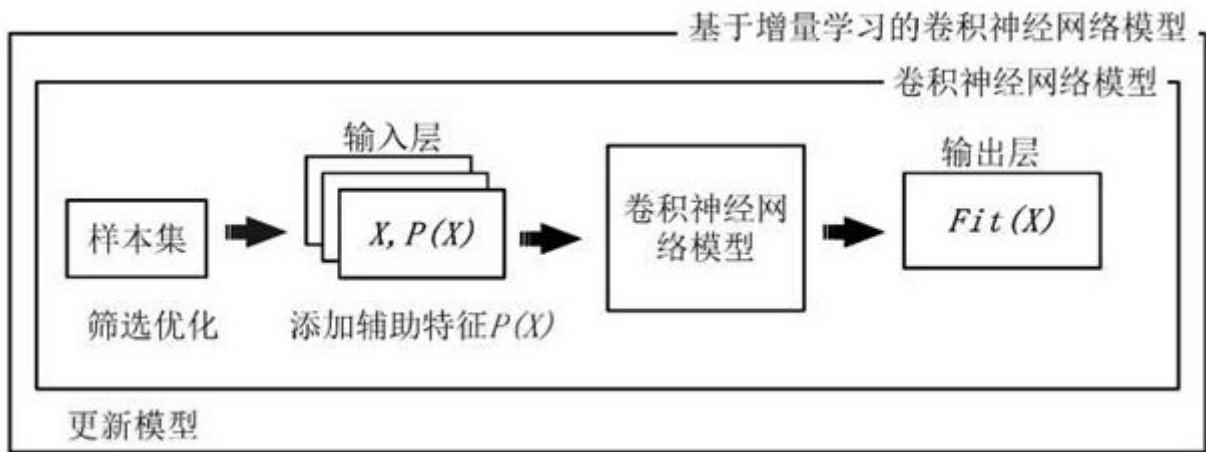


图 2

```

...
if (x > y)
{ int t = x; x = y; y = t; }
if (x > z)
{ int t = x; x = z; z = t; }
if (x+y<=z)
{type=0;}
else{
if(x*x+y+y==z*z)
{type=1;}
else{
if((x==y)&&(y==z))
{type=2;}
else{
if((x==y)||(y==z))
{type=3;}
else
{type=4;}
}
}
}
}
...

```

(a) 源程序

```

...
if (x == y)
{ int t = x; x = y; y = t; }
if (x > z)
{ int t = x; x = z; z = t; }
if (x+y<=z)
{type=0;}
else{
if (x*x+y+y==z*z)
{type=1;}
else{
if((x==y)&&(y==z))
{type=2;}
else{
if((x==y)||(y==z))
{type=3;}
else
{type=4;}
}
}
}
}
...

```

(b) 变异体 M1

```

...
if ((x>y) != (x==y)) { b[0] = 1; } ... // M1
if ((x > y) != (--a1 > y)) { b[1] = 1; } ... // M2
if ((x > y) != (x > ++a2)) { b[2] = 1; } ... // M3
if(x > y) { int t = x; x = y; y = t; }
int a3=x;int a4=y;
if ((x > z) != (x == z)) { b[3] = 1; } ... // M4
if ((x > z) != (--a3 > z)) { b[4] = 1; } ... // M5
if ((x > z) != (x > ++a4)) { b[5] = 1; } ... // M6
if ((x > z) != (x != z)) { b[6] = 1; } ... // M7
if(x > z) { int t = x; x = z; z = t; }
if((x+y<=z) != (x+y!=z)) {b[7]=1;} ... // M8
if((x+y<=z) != (x-y<=z)) {b[8]=1;} ... // M9
if((x+y<=z) != (x/y<=z)) {b[9]=1;} ... // M10
if((x+y<=z) != (x%y<=z)) {b[10]=1;} ... // M11
if(x+y<=z){type=0;}
else {
if((x*x+y+y==z*z) != (x*x+y*y>=z*z)) {b[11]=1;} ... // M12
if((x*x+y+y==z*z) != (x*x+y*y>z*z)) {b[12]=1;} ... // M13
if(x*x+y+y==z*z){type=1;}
else {
int a5=x; int a6=y;int a7=z;
if((x==y)&&(y==z) != (x==y)||(y==z)) {b[13]=1;} ... // M14
if((x==y)&&(y==z)) {type=2;}
else {
int a8=y; int a9=z;
if((x==y)||(y==z) != (x==y)&&(y==z)) {b[14]=1;} ... //M15
if((x==y)||(y==z) != (x==y)||(y<z)) {b[15]=1;} ... // M16
if((x==y)||(y==z) != (x==y)||(y!=z)) {b[16]=1;} ... // M17
if((x==y)||(y==z) != (x==y)||(y>=z)) {b[17]=1;} ... // M18
if((x==y)||(y==z) != (x==y)||(--a8==z)) {b[18]=1;} ... // M19
if((x==y)||(y==z) != (x==y)||(y==++a9)) {b[19]=1;} ... // M20
if((x==y)||(y==z)) {type=3;}
else {type=4;}
}
}
}
}
...

```

(c) 被测程序

图 3

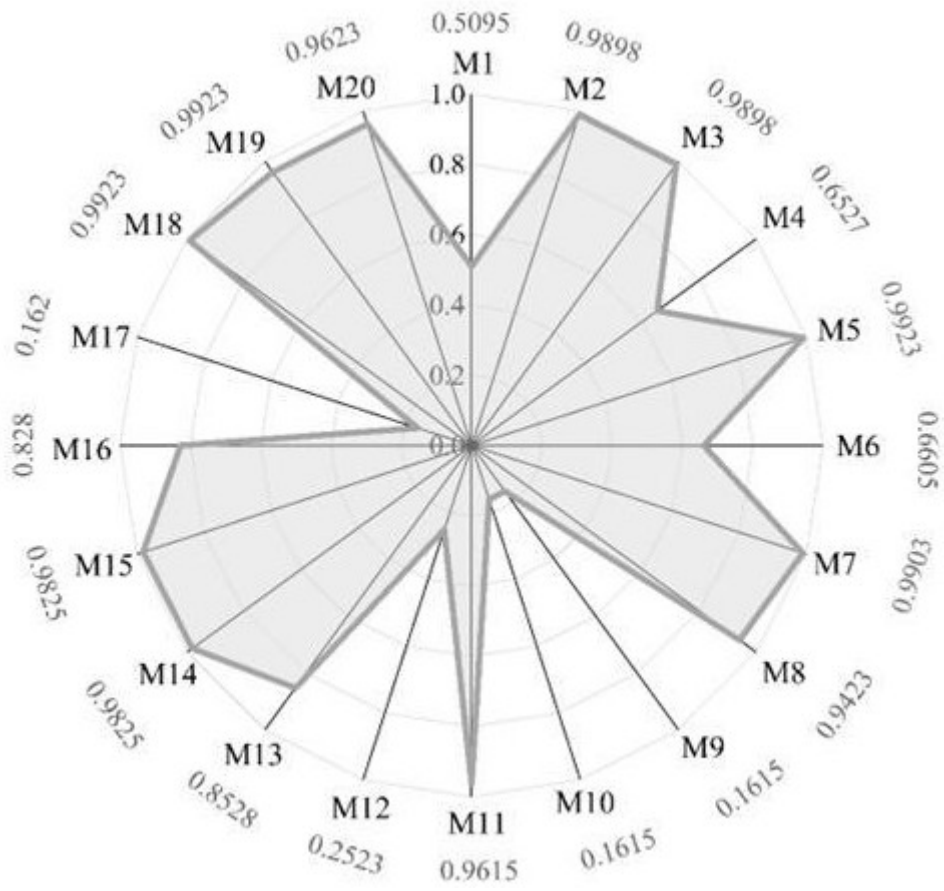


图 4

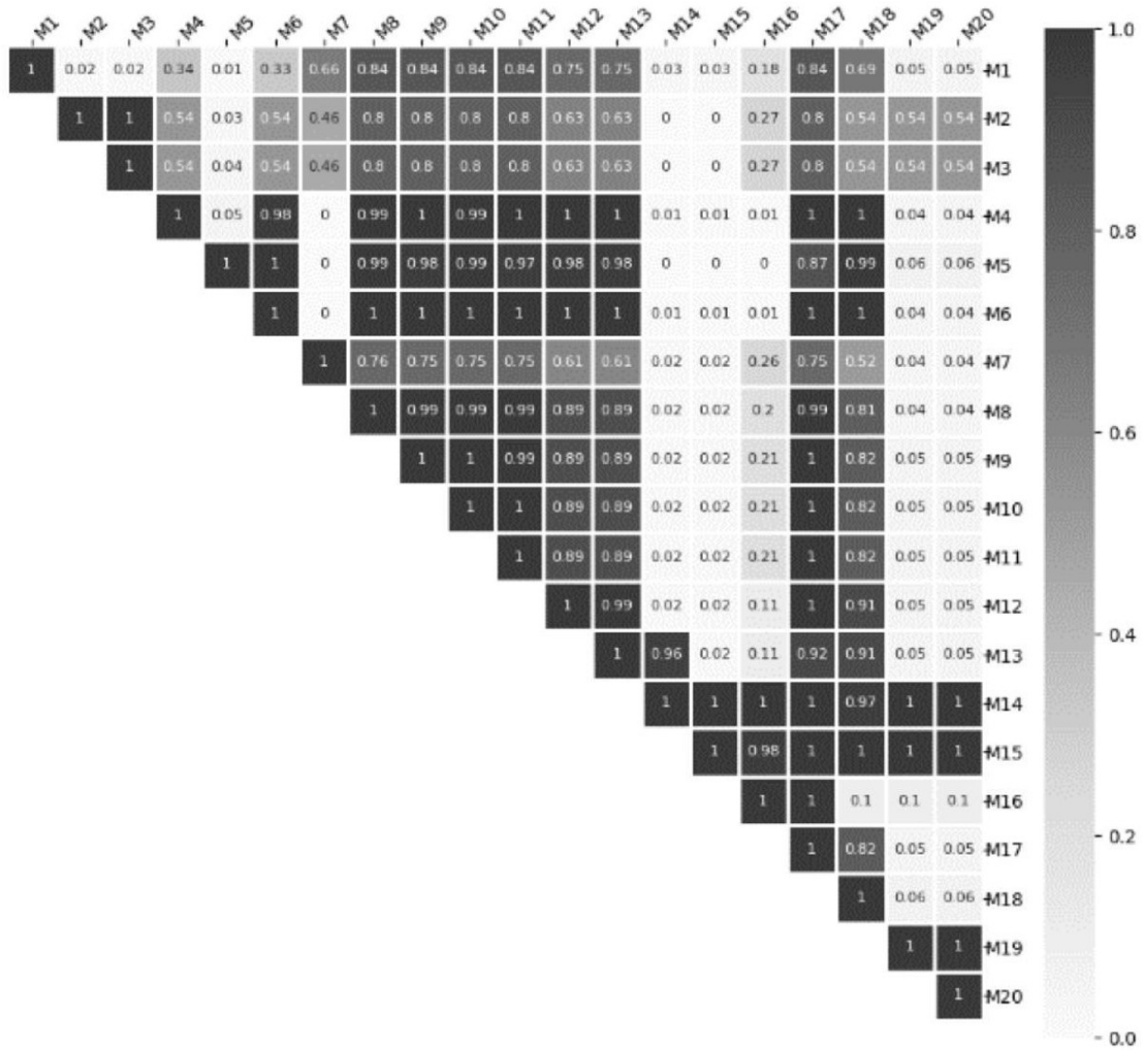


图 5

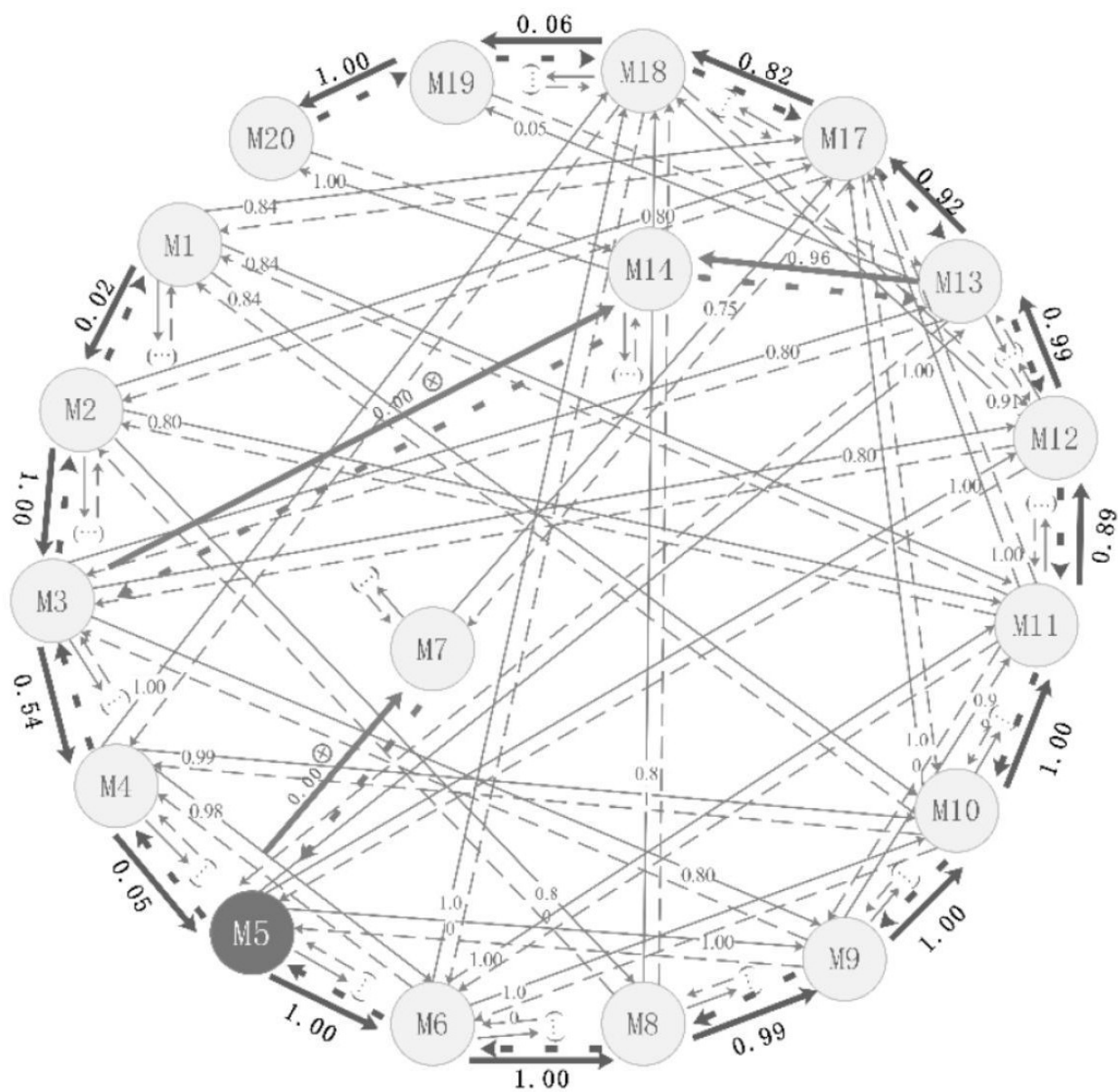


图 6

